

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What is the major difference between the following?

```
import math
```

and

```
from math import sqrt
```

- A)Both import only sqrt
- B)First provides the module namespace; second imports sqrt directly
- C)There is no difference
- D)First imports only sqrt

Q2. What is the output?

```
def fun(x):
    return x * 2
```

```
a = fun
print(a(5))
```

- A)func(5)
- B)5
- C)10
- D>Error

Q3. Why is

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

codes's second handler never reached?

- A)except blocks execute randomly
- B)ZeroDivisionError is not an exception
- C)ZeroDivisionError occurs before Exception
- D)Exception is a superclass of ZeroDivisionError

Q4. What happens here?

```
class MyError(Exception):
```

```
    pass
```

```
def test():
```

```
    raise MyError("Something went wrong")
```

```
try:
```

```
    test()
```

```
except MyError as e:
```

```
    print(type(e).__name__)
```

- A)Exception
- B)Something went wrong
- C)MyError
- D)TypeError

Q5. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

- A)36
- B)NameError
- C)6.0
- D)6

Q6. What is the purpose of this statement?

```
if __name__ == "__main__":
```

- A)It identifies code intended to run when the file is executed directly
- B)It creates the main function
- C)It prevents all imports
- D)It prevents syntax errors

Q7. What is the output?

```
def fun(x):
```

```
    x += 10
```

```
    return x
```

```
x = 5
```

```
y = fun(x)
```

```
print(x, y)
```



A)15 15

B)5 5

C)15 5

D)5 15

Q8. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

```
print(result)
```

A)24

B)0

C)9

D)10

Q9. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

A)random cannot be imported

B)Python will always reject it

C)Python automatically renames the local file

D)The local file can shadow the intended standard-library module

Q10. What will be printed?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        print(x)
```

```
    x = 30  
    inner()
```

```
outer()
```

A)30

B)NameError

C)10

D)20

Q11. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

```
print(fun(3))
```

A)TypeError

B)RecursionError

C)6

D)3

Q12. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

A)10

B)5

C)15

D>Error

Q13. What happens when this function is called?

```
def fun(a=10, b):  
    return a + b
```

A)NameError

B)SyntaxError

C)It executes correctly

D)TypeError

Q14. if test.py has print("A")

```
if __name__ == "__main__":  
    print("B")
```

If test.py is executed directly, what will be printed?

A)Nothing

B)A

C)B

D)A followed by B

Q15. What happens here?

```
def fun(a):  
    return a
```



print(fun())

A)NameError

B)TypeError

C)0

D)None

Q16. What will be printed?

try:

print(10 / 0)

except ZeroDivisionError:

print("A")

except Exception:

print("B")

A)A B

B)Nothing

C)B

D)A

Q17. What happens?

def fun(a, b):

return a + b

print(fun(10, a=20))

A)30

B)SyntaxError

C)20

D)TypeError

Q18. What will be the output?

x = 10

def fun():

print(x)

x = 20

fun()

A)10

B)20

C)NameError

D)UnboundLocalError

Q19. Suppose test.py contains:

print("A")

if __name__ == "__main__":

print("B")

If another Python program executes:

import test

what will normally be printed?

A)A and B

B)Nothing

C)A

D)B

Q20. What will be printed?

class InvalidAgeError(Exception):

pass

def check_age(age):

if age < 18:

raise InvalidAgeError("Age is too low")

return "Allowed"

try:

print(check_age(15))

except InvalidAgeError as e:

print(type(e).__name__)

print(e)

A)Exception

Age is too low

B)InvalidAgeError

Age is too low

C)InvalidAgeError

15

D)Nothing, because custom exceptions cannot be caught

Q21. What is the output?

def fun():

try:

return 10

finally:

return 20

print(fun())

A)20

B)10

C)RuntimeError

D)None



Q22. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)  
print(b)  
A)[10, 10]  
[10]  
B)Error  
C)[10]  
[10, 10]  
D)[10]  
[10]
```

Q23. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

```
A)Error  
B)Only Hello  
C)x  
display  
D)10  
Hello
```

Q24. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

```
A)20  
B)-10  
C)30  
D)10
```

Q25. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        return x + 5
```

```
    return inner
```

```
f = outer()
```

```
print(f())  
A)NameError  
B)10  
C)None  
D)15
```

Q26. What will happen?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal y  
        y = 20
```

```
    inner()
```

```
outer()
```

```
A)SyntaxError  
B)20  
C)10  
D)NameError
```

Q27. What will be printed?

```
try:  
    x = int("10")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

```
A)B C  
B)A C  
C)A  
D)B
```

Q28. What is the output?



```
try:
    x = int("abc")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

A)B C
B)A
C)B
D)A C

Q29. What is the output?

```
def outer():
    x = 10

    def inner():
        nonlocal x
        x = x + 5
        return x

    inner()
    return x

print(outer())
```

A)5
B)15
C)10
D)UnboundLocalError

Q30. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

A)A, followed by propagation of the same exception
B)B
C)A, and execution ends normally
D)No Output

Q31. What is the output?

```
try:
    print(10 / 0)
except Exception:
    print("A")
```

```
except ZeroDivisionError:
    print("B")
```

A)A B
B)B
C)SyntaxError
D)A

Q32. Which object keeps track of modules that have already been imported during the current Python process?

A)sys.imports
B)sys.modules
C)sys.path
D).__module__

Q33. What will be printed?

```
x = 10

f = lambda: x

x = 20

print(f())
```

A)20
B)10
C)None
D)NameError

Q34. What is the output?

```
numbers = [1, 2, 3, 4, 5]

result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
            numbers)
    )
)

print(result)
```

A)[2, 4]
B)[1, 3, 5]
C)[4, 8]
D)[2, 4, 6, 8, 10]

Q35. What is the output?

```
def fun(x):
    if x == 0:
```



```
return
return x + fun(x - 1)
```

print(fun(3))

- A)6
- B)3
- C)None
- D)5

Q36. What is the output?

```
def fun():
    try:
        print("A")
        raise ValueError
    except ValueError:
        print("B")
        return 10
    else:
        print("C")
        return 20
    finally:
        print("D")
```

print(fun())

- A)A
- C
- D
- 20
- B)A
- B
- D
- 10
- C)A
- B
- C
- D
- 10
- D)A
- B
- 10
- D

Q37. What will be printed?

```
def make_counter():
    count = 0

    def counter():
```

```
nonlocal count
count += 1
return count
```

return counter

```
a = make_counter()
b = make_counter()
```

```
print(a())
print(a())
print(b())
print(a())
```

- A)1
- 2
- 1
- 3
- B)0
- 1
- 0
- 2
- C)1
- 1
- 1
- 1
- D)1
- 2
- 3
- 4

Q38. What will be printed?

```
def fun(a, b):
    return a + b
```

print(fun(10, b=20))

- A)20
- B)30
- C)TypeError
- D)SyntaxError

Q39. What will be the output?

```
def fun(x):
    try:
        if x < 0:
            raise ValueError("negative")
        return x
    except ValueError:
        return 100
```



```
finally:  
    print("cleanup")
```

```
print(fun(-5))
```

- A)100
- cleanup
- B)cleanup
- 5
- C)cleanup
- 100
- D)negative
- cleanup

Q40. What happens here?

```
try:  
    raise ValueError("Wrong value")  
except ValueError as e:  
    print(e)
```

- A)e
- B)ValueError
- C)Wrong Value
- D)Nothing

Q41. What will happen?

```
def fun(a, b=10):  
    return a + b
```

```
print(fun(b=5, a=20))
```

- A)SyntaxError
- B)15
- C)25
- D)TypeError

Q42. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

- A)Module loaded
- 10
- B)ImportError
- C)10
- 10
- D)Module loaded
- Module loaded
- 10

Q43. What will normally happen?

```
import math  
import math
```

- A)SyntaxError
- B)The module is normally loaded and executed twice
- C)The module is normally loaded once and reused
- D)ImportError

Q44. What does the following function return?

```
def fun():  
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

- A>Hello None
- B>Error
- C)None
- D>Hello

Q45. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

```
variable = 10
```

```
fun()
```

- A)50
- B)10
- C)NameError
- D)UnboundLocalError

Q46. What happens when this code runs?

```
def first():
```



second()

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

```
try:  
    first()  
except RuntimeError:  
    print("Handled")  
A)NameError  
B)Handled  
C)RuntimeError cannot propagate through  
functions  
D)TypeError
```

Q47. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)25
B)5
C)sqrt(25)
D)5.0

Q48. What will be the output?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        global x  
        x = 30
```

```
    inner()  
    print(x)
```

```
outer()  
print(x)
```

A)20
30
B)20
10
C)30
30

D)30
20

Q49. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

fun()

A)A
B)Nothing
C)A B
D)B

Q50. What will be printed?

```
def fun():  
    try:  
        return 10  
    finally:  
        print("Finally")
```

```
print(fun())
```

A)10 only
B)Finally only
C)Finally followed by 10
D)10 followed by Finally

